

De l'ontologie au ERD

Construire le bon modèle de données

FDS

UEH

Langage Ubiquitaire · Niveaux d'abstraction · Entités · ERD · Normalisation · SQL/NoSQL

1. Communication

2. Vocabulaire

3. ERD

4. Normalisation

5. SQL/NoSQL

6. SQL + SoR

Jean Fritz SAINT-PAUL

Plan du cours

Semaine 8, Cours 1

1 10 min	La modélisation comme acte de communication Langage Ubiquitaire (DDD) · 3 niveaux d'abstraction · lien avec la Semaine 7
2 15 min	Identifier les entités depuis vos User Stories Fait-Based Modeling · noms et verbes · table US → entités · règle MVP
3 15 min	Construire le ERD (modèle logique brut) 4 composants · cardinalités · clés PK/FK · ERD complet FDS SYS
4 10 min	Normaliser le modèle 3 anomalies · 1NF / 2NF / 3NF / BCNF · vérification FDS SYS
5 5 min	Choisir SQL ou NoSQL ACID vs BASE · familles de bases de données · règle de décision
6 5 min	Du ERD normalisé au SQL + System of Record Structure CREATE TABLE · contraintes · source de vérité inter-modules

1

La modélisation comme acte de communication

Langage Ubiquitaire (DDD) · 3 niveaux d'abstraction · lien avec la Semaine 7

1 — La modélisation comme acte de communication

Pourquoi le vocabulaire précède le SQL

Communication

Question : avant d'écrire CREATE TABLE, est-ce que toute votre équipe est d'accord sur ce qu'est un « Étudiant » dans votre système ?

Le vrai risque

La modélisation des données n'est pas qu'une question de stockage.

Le plus grand danger : deux développeurs avec deux définitions différentes du même concept.

Résultat : un système incohérent qui ne peut plus évoluer sans casser quelque chose.

La solution : Langage Ubiquitaire

Un vocabulaire commun entre développeurs et experts métier, défini une fois, utilisé partout :

- ✓ dans le code
- ✓ dans les discussions
- ✓ dans les tests
- ✓ dans la documentation

Un contrat vivant que toute l'équipe respecte.

"Bad programmers worry about the code. Good programmers worry about data structures and their relationships." — Linus Torvalds

1 — Langage Ubiquitaire — Exemples FDS

Définir chaque terme une fois, pour toute l'équipe

Communication

Un Langage Ubiquitaire n'est pas un glossaire passif mais un contrat vivant.

Étudiant

Un utilisateur avec `role = 'student'` et un accès actif à au moins un module. Ce n'est pas un alumnus (ancien de la fac), ce n'est pas un prospect, mais c'est quelqu'un qui peut se connecter aujourd'hui.

Accès

La relation entre un Étudiant et un Module, avec une `granted_at` (date d'octroi) et une révocation possible (`revoked_at`).

Désactivé

Un utilisateur dont le `status = 'disabled'` — il ne peut plus se connecter, mais ses données sont conservées. Ce n'est pas une suppression.

1 — Les 3 niveaux d'abstraction d'un modèle de données

Tout modèle passe par ces 3 niveaux — dans l'ordre

Communication

1

Conceptuel

Qu'est-ce que le système doit connaître ?

Audience

Chefs de projet · experts domaine · analystes

Livrable

Glossaire des entités · ERD haut niveau

2

Logique

Comment ces concepts sont-ils structurés et reliés ?

Audience

Architectes · développeurs senior

Livrable

ERD normalisé · PK/FK définis

3

Physique

Comment cela est-il stocké concrètement ?

Audience

DBA · DevOps

Livrable

Scripts SQL (DDL) · stratégie d'index

Semaine 8 : vous traversez ces 3 niveaux dans l'ordre — vocabulaire (conceptuel) → ERD normalisé (logique) → CREATE TABLE (physique)

2

Identifier les entités depuis vos User Stories

Fait-Based Modeling · noms et verbes · table US → entités · règle MVP

2 — Phase vocabulaire : noms et verbes

Fact-Based Modeling · verbaliser avant de modéliser

Vocabulaire

Principe : On ne commence jamais par les tables. On commence par les faits.

Pour chaque User Story Must Have, écrivez 1 à 3 faits élémentaires en langage naturel :

1

"Un Administrateur **invite** un Étudiant par email."

Noms → entités candidates : Administrateur · Étudiant · Email

2

"Un Étudiant **se connecte** et reçoit un token JWT."

Noms → entités candidates : Étudiant · JWTToken

3

"Un Administrateur **accorde** un Accès à un Étudiant pour un Module."

Noms → entités candidates : Administrateur · Module · Access

4

"Un Administrateur **désactive** un compte Étudiant."

Noms → entités candidates : Administrateur · compte Étudiant

Dans chaque phrase : les NOMS sont des entités candidates · les VERBES sont des relations candidates

2 — De la User Story à l'entité

Chaque User Story Must Have révèle des entités et des relations

Vocabulaire

User Story	Entités identifiées	Attributs clés
US-P1 — Email d'invitation	Etudiant · Administrateur · Invitation	email, token, expires_at, used_at
US-P2 — Connexion SSO	Etudiant · Administrateur · JWTToken	password_hash, status, token_hash, expires_at
US-P3 — Accès révoqué	Etudiant · Administrateur	status (active/disabled), disabled_at
US-J1 — Import CSV	Etudiant · Administrateur · Access · Module	user_id, module_id, granted_at, granted_by
US-J2 — Désactivation compte	Etudiant · Administrateur · Access	status, revoked_at
US-J3 — Tableau de bord	Etudiant · Administrateur · Access · Module	(lecture seule — aucune nouvelle table)

6 entités retenues pour le MVP : Etudiant · Administrateur · Module · Access · Invitation · JWTToken

2 — Règle MVP : la contrainte de sobriété

Ne modéliser que ce qui est nécessaire pour le Walking Skeleton

Vocabulaire

Si une entité n'est référencée par aucune User Story Must Have, elle n'est pas dans le MVP.

À chaque étape, posez la question :

"Est-ce que cette table est nécessaire pour livrer le Walking Skeleton ?"

Entités retenues pour FDS SYS — MVP :

Etudiant	Admin	Module	Access	Invitation	JWTToken
Identité · statut	Identité · statut	Nom · slug · URL	Jointure N-N enrichie	Email · token · expiration	Session · révocation

Cette discipline s'applique tout au long du processus : vocabulaire → ERD → normalisation → SQL.
Si vous créez une table que le Walking Skeleton n'utilise pas, vous sur-engineez votre modèle.

3

Construire le ERD (modèle logique brut)

4 composants · cardinalités · clés PK/FK · ERD complet FDS SYS

3 — Les 4 composants d'un ERD

Entity-Relationship Diagram · la carte de vos données

ERD

Une fois les entités identifiées, on construit le ERD, la carte des données, avant de parler de SQL.

1

Entité

Règle : si vous stockez des informations SUR quelque chose, c'est une entité.

Exemples : Etudiant, Administrateur, Module, Access, Invitation, JWTToken

Représentée par un rectangle dans le diagramme. Correspond souvent à une table SQL.

Entité

2

Attribut

Règle : chaque attribut décrit UNIQUEMENT l'entité à laquelle il appartient.

Types courants : UUID · VARCHAR · INTEGER · BOOLEAN · TIMESTAMP · ENUM

Correspond à une colonne dans la table SQL. email appartient à User, pas à Access.

Attribut

3

Relation

Règle : une relation relie deux entités et indique combien d'instances peuvent participer.

1—1 (profil unique) · 1—N (plusieurs tokens) · N—N (étudiants × modules)

La cardinalité détermine le type d'implémentation SQL : FK, FK UNIQUE, ou table de jointure.

Relation

4

Clés

Règle : PK = identifiant unique et immuable. FK = traduction d'une relation.

PK : UUID (systèmes distribués) · FK : référence vers la PK d'une autre entité

Utilisez UUID comme PK pour les systèmes distribués. Les FK créent l'intégrité référentielle.

Clés

3 — Cardinalités et clés

Comment les entités se connectent · comment les identifier en SQL

ERD

Cardinalité	Notation	Signification	Implémentation SQL
1 — 1	—○ ○ —	Un utilisateur a un seul profil	Clé étrangère UNIQUE
1 — N	—○ ○{—	Un utilisateur a plusieurs tokens JWT	Clé étrangère côté « plusieurs »
N — N	—○{○{—	Un étudiant dans plusieurs modules, un module avec plusieurs étudiants	Table de jointure intermédiaire (Access)

Clés — fondations de l'intégrité relationnelle

Clé Primaire (PK)

- Identifiant unique et immuable de chaque ligne
- Utilisez UUID pour les systèmes distribués
- Ex : `id UUID PRIMARY KEY DEFAULT gen_random_uuid()`

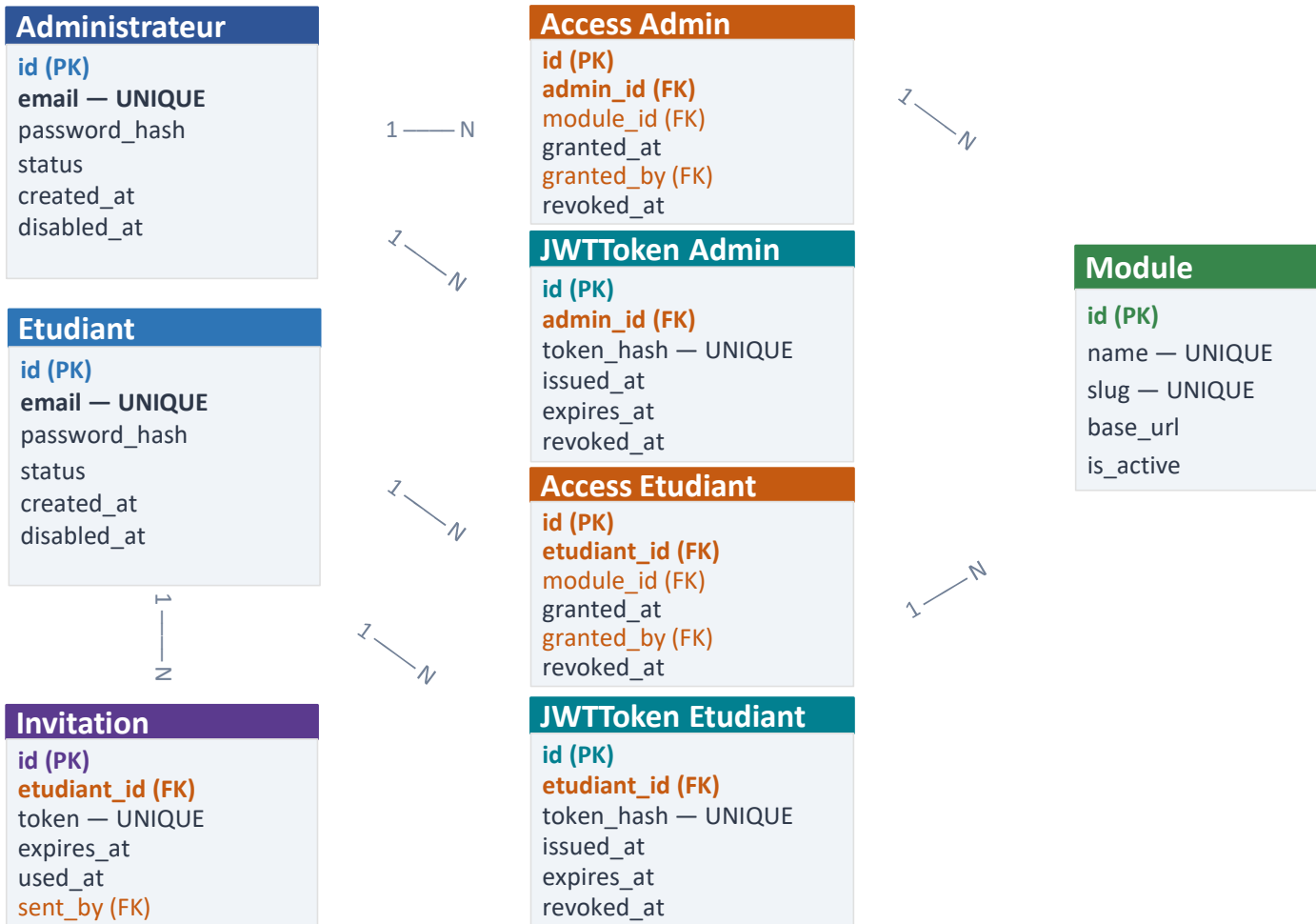
Clé Étrangère (FK)

- Référence vers la PK d'une autre entité
- Traduction directe d'une relation dans le ERD
- Ex : `user_id UUID REFERENCES users(id)`

3 — ERD brut FDS SYS (avant normalisation)

6 entités · attributs · relations · cardinalités

ERD



Access = jointure N-N
User x Module + métadonnées

JWTToken séparé de User
plusieurs sessions + révoc.

Jamais 'password'
— toujours password_hash

4

Normaliser le modèle

3 anomalies · 1NF / 2NF / 3NF / BCNF · vérification FDS SYS

4 — Normalisation : les 3 anomalies que l'on prévient

Pourquoi un ERD brut ne suffit pas

Normalisation

Un ERD brut peut contenir des redondances qui, en production, créent des incohérences irréparables.

Anomalie d'insertion

Impossible d'ajouter un fait sans en ajouter d'autres.

Si l'URL d'un module est stockée dans chaque ligne d'Access, on ne peut pas créer un module sans lui affecter immédiatement un étudiant.

Solution : séparer Module en table indépendante avec sa propre PK.

Anomalie de modification

Changer un fait exige de mettre à jour des dizaines de lignes — avec risque d'incohérence.

L'URL d'un module change → il faut la modifier dans toutes les lignes d'Access. Une seule ligne oubliée = données contradictoires.

Solution : stocker base_url UNE SEULE FOIS dans la table Module.

Anomalie de suppression

Supprimer un fait efface accidentellement d'autres informations.

Supprimer le dernier accès d'un étudiant supprime la trace de l'existence du module — si le module n'a pas sa propre table.

Solution : chaque entité dans sa propre table — une suppression n'affecte que ce qu'elle doit affecter.

Bloc 4 — Les formes normales : progression cumulative

1NF → 2NF → 3NF → BCNF · chaque niveau élimine une anomalie

Normalisation

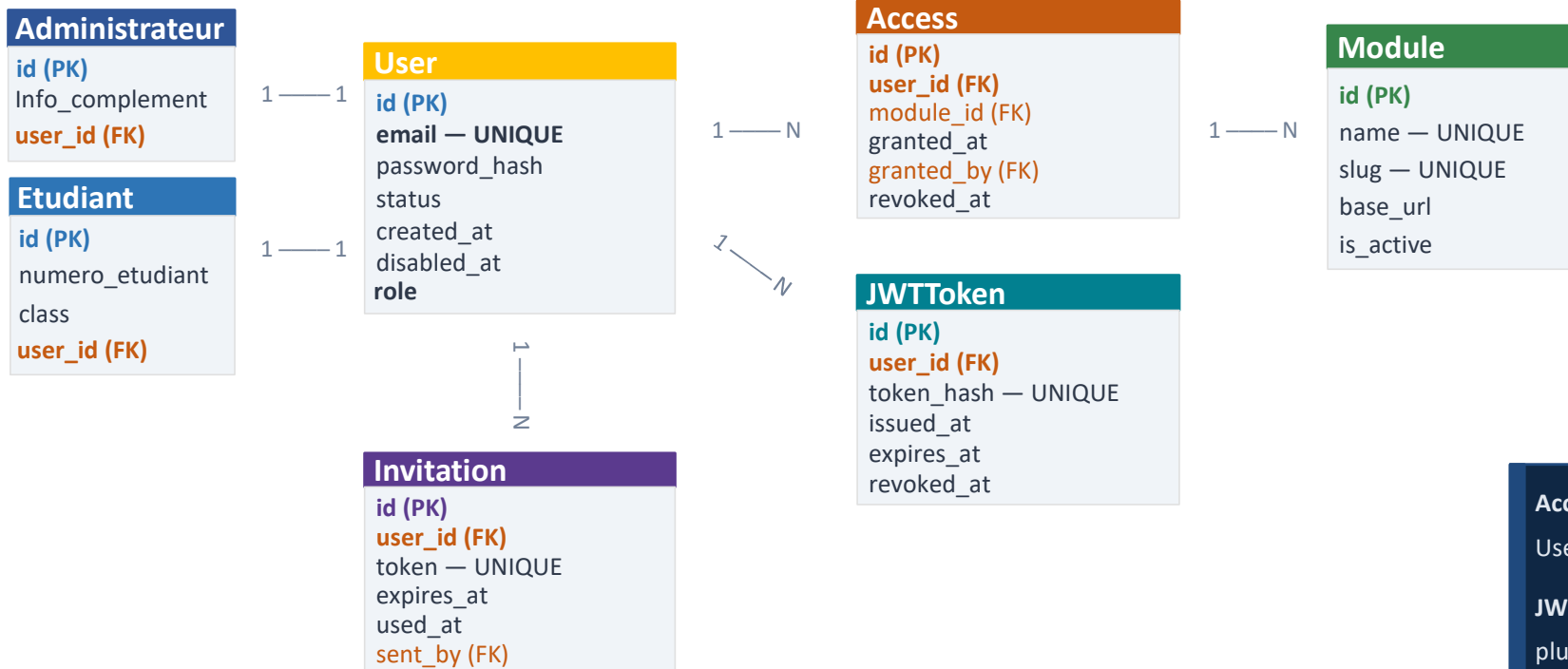
Forme	Règle	Anomalie éliminée
1NF	Chaque attribut contient une valeur atomique — pas de listes, pas de groupes répétés.	Valeurs multi-valuées (ex : modules: "Akademi, Pay" dans une seule colonne)
2NF	Tout attribut non-clé dépend de la TOTALITÉ de la clé primaire.	Dépendance partielle (problème avec les clés composites — sans impact avec UUID PK)
3NF	Aucun attribut non-clé ne dépend d'un autre attribut non-clé.	Dépendance transitive (ex : nom_etudiant stocké dans la table Access)
BCNF Boyce–Codd normal form	Tout déterminant est une clé candidate.	Anomalies résiduelles (quand plusieurs clés candidates se chevauchent)

Standard MVP : 3NF est le minimum exigé. Règle pratique : si vous stockez le même fait à deux endroits, votre schéma n'est pas normalisé.

4 — ERD FDS SYS (après normalisation)

7 entités · attributs · relations · cardinalités

ERD



Access = jointure N-N
User x Module + métadonnées

JWTToken séparé de User
plusieurs sessions + révoc.

Jamais 'password'
— toujours password_hash

4 — Vérification : le ERD FDS SYS est déjà en 3NF

Un bon travail de vocabulaire produit un modèle déjà normalisé

Normalisation



1NF

Tous les attributs sont atomiques — aucune liste dans une colonne.

email est une valeur simple, pas un tableau d'emails. status est un ENUM, pas une liste.



2NF

Les clés primaires sont des UUID simples — aucune clé composite.

Sans clé composite, les dépendances partielles sont impossibles par construction.



3NF

Aucune dépendance transitive — chaque donnée est dans la bonne table.

email et nom sont dans User, pas dans Access. base_url est dans Module, pas dans Access.

Le ERD FDS SYS est déjà en 3NF — c'est le résultat d'avoir bien fait la phase vocabulaire.

5

Choisir SQL ou NoSQL

ACID vs BASE · familles de bases de données · règle de décision

5 — ACID vs BASE : le vrai critère de décision

Le choix technologique vient APRÈS la modélisation

SQL / NoSQL

Le choix de la technologie de stockage vient APRÈS la modélisation — pas avant. C'est votre modèle qui dicte le choix.

Critère	SQL (relationnel)	NoSQL (non-relationnel)
Schéma	Prédéfini, rigide	Dynamique, flexible
Cohérence	ACID — Atomicité, Cohérence, Isolation, Durabilité	BASE — Cohérence éventuelle (Eventually Consistent)
Scalabilité	Verticale (serveurs plus puissants)	Horizontale (plus de nœuds)
Cas d'usage	Transactions · droits d'accès ERP · données critiques	Big Data · analytics temps réel données non structurées

ACID : chaque transaction réussit entièrement ou échoue entièrement, et la base reste dans un état valide. Indispensable pour les droits d'accès : un compte désactivé ne doit jamais avoir un token valide, même pendant 30 ms.

BASE : cohérence éventuelle en échange de scalabilité massive — acceptable pour des fils d'actualité, pas pour des permissions.

5 — Familles de bases de données + règle de décision

5 familles · 2 questions · 1 décision justifiée

SQL / NoSQL

Famille	Exemples	Structure	Point fort
Relationnelle (SQL)	PostgreSQL · MySQL · SQLite	Tables, lignes, colonnes	Relations complexes · cohérence ACID
Document (NoSQL)	MongoDB · Firebase	JSON / BSON imbriqué	Flexibilité de schéma · données hiérarchiques
Clé-Valeur	Redis · DynamoDB	Paires clé→valeur	Vitesse · cache · sessions
Colonne large	Cassandra · HBase	Colonnes dynamiques par ligne	Volume massif · écritures rapides
Graphe	Neo4j · TypeDB	Nœuds · relations · types	Relations très complexes · inférence logique

Choisir SQL si :

relations claires et bien définies · cohérence forte (ACID) requise · schéma stable

Choisir NoSQL (document) si :

schéma évolue fréquemment · données naturellement hiérarchiques · volumes massifs

Plateforme FDS : relations fortes + cohérence ACID exigée = PostgreSQL — Le choix est justifié par le modèle, pas par la mode.

6

Du ERD normalisé au SQL + System of Record

Structure CREATE TABLE · contraintes · source de vérité inter-modules

6 — Du ERD normalisé au SQL

Structure d'un CREATE TABLE · contraintes essentielles

SQL + SoR

```
CREATE TABLE nom_table (  
  id          UUID          PRIMARY KEY  
              DEFAULT gen_random_uuid(),  
  
  col1        TYPE          NOT NULL,  
  col2        TYPE          DEFAULT valeur,  
  
  fk_id       UUID  
              REFERENCES autre_table(id)  
              ON DELETE CASCADE,  
  
  created_at  TIMESTAMP DEFAULT NOW()  
);
```

```
-- Exemple FDS SYS  
CREATE TABLE users (  
  id          UUID          PRIMARY KEY  
              DEFAULT gen_random_uuid(),  
  email       VARCHAR(255) UNIQUE NOT NULL,  
  role        VARCHAR(20)  NOT NULL,  
  status      VARCHAR(20)  DEFAULT 'active'  
);
```

NOT NULL

Valeur obligatoire — impossible d'insérer sans cette colonne

UNIQUE

Pas de doublon — appliqué à email, slug, token_hash

REFERENCES

Clé étrangère — intégrité référentielle garantie par la BDD

ON DELETE CASCADE

Supprimer le parent supprime aussi les enfants (Access, JWTToken)

ON DELETE RESTRICT

Interdit la suppression si des enfants existent — plus prudent

CHECK (...)

Validation de domaine en base CHECK (role IN ('student','admin'))

CREATE INDEX

Accélère les colonnes utilisées dans WHERE et JOIN (user_id, module_id)

6 — System of Record : source de vérité inter-modules

Qui possède la donnée ? Qui la consomme ?

SQL + SoR

Le System of Record est le module propriétaire d'une donnée — la seule source de vérité pour cette donnée. Si FDS SYS possède l'identité (User), FDS Akademi ne duplique pas cette table — il dépend de l'interface exposée par SYS.

Donnée	Propriétaire (System of Record)	Modules dépendants
Identité User	FDS SYS	FDS Akademi · FDS Pay · FDS Portail · FDS Admin
Liste Module	FDS SYS	Tous les modules de la plateforme
Access (droits d'accès)	FDS SYS	Tous les modules (vérification JWT)

Chaque équipe doit identifier :

- 1 Quelles données son module POSSÈDE — il en est le System of Record
- 2 Quelles données son module CONSOMME depuis un autre module — dépendance d'interface

Votre modèle de données est complet quand vous pouvez répondre à ces 2 questions pour chaque donnée de votre module.

Lectures recommandées — Cours 1

5 articles fondamentaux · ~60 minutes au total · à lire avant ou après le cours

5 min	Martin Fowler — Ubiquitous Language
Partie 1	martinfowler.com/bliki/UbiquitousLanguage.html — La définition originale du Langage Ubiquitaire. À lire absolument avant le cours.
15 min	DataCamp — Data Modeling Explained
Partie 1	datacamp.com/blog/data-modeling — Vue d'ensemble des niveaux conceptuel, logique et physique avec des exemples concrets.
15 min	Agile Data — Data Modeling 101
Partie 2–3	agiledata.org/essays/datamodeling101.html — Introduction pratique : entités, attributs, relations, cardinalités avec exemples visuels.
15 min	Codecademy — Normalization in DBMS
Partie 4	codecademy.com/article/normalization-in-dbms — 1NF à BCNF avec exemples pas à pas et contre-exemples — prépare l'exercice de normalisation.
10 min	IBM — SQL vs. NoSQL Databases
Partie 5	ibm.com/think/topics/sql-vs-nosql — Trade-offs ACID/BASE, cas d'usage de chaque famille, critères de décision. Comparatif structuré.